



Segmentation Faults

A huge amount of application code is developed every moment—hence, program failures are very common. Often a program ends abruptly, leaving no clue as to why it failed. When it is accompanied by the famous ‘segmentation fault’ message, users are lost. This article helps readers understand what happens behind the scenes, and exactly where the application fails.

Let’s begin by understanding what the term ‘core dump’ means and discovering how it can help you. A core dump is the memory snapshot of a process at a specific time, especially when the process crashes (exits abnormally). The major pieces of the program state are dumped, including registers and memory management information. Core dumps assist in debugging errors in the program. The memory segments of a process have text, data, heap and stack.

In order to get core dumps, you need to enable the kernel configuration `CONFIG_ELF_CORE` in the running kernel. If the running kernel has this configuration disabled, enable it, rebuild the kernel, and boot the system with the new kernel. Second, ensure that the core dump is turned on—the core of a crashed process will be dumped if the core file size is set. To do this, use the `ulimit` command. The `-c` option gives the current size setting for core files; `ulimit -c 0` turns off core dumps, while `ulimit -c x` sets the size of the core file to `x` (e.g., 1024) bytes. And `ulimit -c unlimited` turns on the core dump with an unlimited size for the core file.

When an illegal statement is executed, the process receives a signal. If the default action of the signal is to dump the core before exit, it checks for the user limit set for the core file. If the set value turns on the core dump, it

dumps the core file. When the program receives signals, it performs the default action of the signal, unless the user has registered other functions as signal handlers, in the program. Some signals on Linux (whose default action is to dump the core before exiting) are `SIGQUIT`, `SIGILL`, `SIGABRT`, `SIGFPE`, `SIGSEGV`, `SIGBUS`, `SIGSYS`, `SIGTRAP`, `SIGXCPU` and `SIGXFSZ`.

Triggering a seg-fault deliberately

You can deliberately include an illegal statement in a program—dereferencing a `NULL` pointer, as shown in the following code (call it `test.c`):

```
#include <stdio.h>
int main()
{
    int *ptr = NULL;
    printf ("Dereferencing the NULL pointer: %d/n", *ptr);
    return 0;
}
```

The value of `ulimit -c` has been set to `unlimited` so when you compile the code with `gcc -g -o test test.c` and then run the compiled program, you will receive